

Lista V — Python (versão adaptada e ampliada da Lista IV)

Aviso: este é um documento alterado, gerado a partir da Lista IV original (em C). Os exercícios foram adaptados para a linguagem Python e novos exercícios foram acrescentados. Não é um material oficial da disciplina.

Parte 1 — Exercícios adaptados da lista original

1. Escreva um programa em Python que imprima todos os números de 100 até 1.
2. Escreva um programa em Python que imprima a soma dos números pares entre 25 e 525.
3. Escreva um programa em Python que leia vários números inteiros até ser lido o zero. O programa deve informar quantos números digitados estavam entre 100 e 200.
4. Escreva um programa em Python que leia o nome, a idade e o salário de 20 pessoas (armazenando os dados em listas ou em uma lista de dicionários). Imprima o nome das pessoas que tiverem salário maior que dois mil reais.
5. Escreva um programa em Python que leia a idade de um grupo de 20 pessoas. O programa deve determinar a idade da pessoa mais velha, a idade da mais nova e a média das idades do grupo.
6. Escreva um programa em Python que gere os trinta primeiros termos da série de Fibonacci: 1, 1, 2, 3, 5, 8, 13, ...
7. **O jogo do marciano.** Um marciano chegou a uma floresta e se escondeu atrás de uma das 100 árvores quando viu um caçador. O caçador só tinha cinco balas em sua espingarda. Cada vez que ele atirava e não acertava, o marciano dizia: “estou mais para a direita” ou “estou mais para a esquerda”. Se o caçador não conseguir acertar o marciano em cinco tiros, ele é levado para Marte. Implemente este jogo em Python para dois jogadores, onde um escolhe a árvore (1 a 100) em que o marciano se esconde e o outro tenta acertar.

Desafio extra: modifique o programa para que o caçador jogue sozinho contra o computador, usando busca binária para tentar acertar o marciano no menor número de tiros possível (isso deve dar sempre certo em até 7 tentativas, já que $2^7 > 100$). Compare o desempenho de uma estratégia de busca binária com uma estratégia de “chute aleatório”.

Parte 2 — Exercícios adicionais (nível intermediário)

8. **Verificador de números primos.** Escreva um programa que leia um número inteiro e informe se ele é primo. Em seguida, gere e imprima todos os números primos entre 1 e 1000.
9. **Contador de vogais e consoantes.** Leia uma frase digitada pelo usuário e conte quantas vogais, consoantes, espaços e outros caracteres ela possui.

10. **Palíndromo.** Escreva um programa que verifique se uma palavra ou frase é um palíndromo (ignorando espaços, acentos e diferença entre maiúsculas/minúsculas). Ex.: “arara”, “A base do teto desaba”.

11. **Números perfeitos.** Um número é chamado “perfeito” quando a soma de seus divisores próprios (excluindo ele mesmo) é igual a ele. Ex.: $6 = 1 + 2 + 3$. Escreva um programa que encontre todos os números perfeitos entre 1 e 10.000.

12. **Matriz mágica.** Leia uma matriz quadrada de ordem N (o usuário escolhe N) e verifique se ela é um “quadrado mágico”, ou seja, se a soma dos elementos de cada linha, cada coluna e das duas diagonais principais é a mesma.

13. **Criptografia de César.** Escreva um programa que criptografe uma mensagem digitada pelo usuário usando a cifra de César (deslocamento de letras no alfabeto), com um número de deslocamento escolhido pelo usuário. Escreva também a função que descriptografa a mensagem.

Parte 3 — Exercícios instigantes e um pouco mais complexos

14. **Torre de Hanói.** Implemente o clássico problema da Torre de Hanói de forma recursiva, para N discos (o usuário escolhe N). O programa deve imprimir cada movimento necessário para resolver o problema (ex.: “Mover disco 1 da torre A para a torre C”) e, ao final, informar o número total de movimentos realizados. Compare esse número com a fórmula teórica $2^N - 1$.

15. **Jogo da Velha (Tic-Tac-Toe) para dois jogadores.** Implemente o jogo da velha no terminal, com um tabuleiro 3x3, alternando jogadas entre “X” e “O”, detectando automaticamente vitória, empate e jogadas inválidas (posição já ocupada).

Desafio extra: implemente um modo “jogador vs. computador”, em que o computador joga de forma pelo menos razoável (ex.: bloqueia o adversário quando ele está prestes a vencer).

16. **Detetive dos números (jogo de adivinhação com pistas).** O computador sorteia um número secreto entre 1 e 1000. O jogador tenta adivinhar, e a cada tentativa o programa informa se o número secreto é “maior” ou “menor” — mas com uma pegadinha: a cada 3 tentativas erradas, o programa pode “mentir” uma vez (o jogador não sabe quando). O jogador vence se acertar em até 12 tentativas, considerando que pode ter recebido uma pista falsa. (Esse exercício trabalha lógica condicional e controle de estado.)

17. **Simulador de labirinto (busca em grade).** Represente um labirinto como uma matriz de 0s (caminho livre) e 1s (parede). Dado um ponto de partida e um ponto de chegada, escreva um programa que determine se existe um caminho entre eles (usando busca em profundidade ou em largura) e, se existir, imprima o caminho encontrado.

18. **O problema dos discos de Fibonacci-Marciano (combinado).** Uma variação criativa do jogo do marciano (exercício 7): agora existem dois marcianos escondidos em árvores diferentes dentre 100 árvores, e o caçador tem 8 balas. As dicas de “direita”/“esquerda” sempre se referem ao marciano mais próximo do tiro. O jogador deve deduzir a posição dos dois marcianos e acertar ambos. Pense em como estruturar as pistas e a lógica — esse é um bom exercício de raciocínio lógico e

estruturação de dados.

19. **Números de Armstrong / narcisistas.** Um número narcisista de N dígitos é igual à soma de seus próprios dígitos elevados à potência N (ex.: $153 = 1^3 + 5^3 + 3^3$). Escreva um programa que encontre todos os números narcisistas de 3 dígitos, depois generalize para números de N dígitos.

20. **Simulador de roleta de probabilidade (Monte Carlo).** Use simulação para estimar o valor de π : gere pontos aleatórios dentro de um quadrado que contém um círculo inscrito e calcule a proporção de pontos que caem dentro do círculo. Quanto mais pontos, mais precisa a estimativa. Mostre como o erro diminui conforme o número de simulações aumenta.

Observações

- Os exercícios da Parte 1 seguem fielmente a lista original, apenas adaptados à sintaxe e aos idiomas do Python (uso de `input()`, listas, `for/while`, funções, etc.).
- Os exercícios das Partes 2 e 3 foram pensados para explorar recursão, estruturas de dados (listas, matrizes, dicionários), lógica de jogos, aleatoriedade e um pouco de algoritmos de busca — bons para quem já domina o básico e quer um próximo desafio.